Measure Theory & Semantics of Probabilistic Programming

Xin Zhang
Peking University

Most of the content is from "Semantics of Probabilistic Programming:
A Gentle Introduction" by Fredrik Dahlqvist, Alexandra Silva, and Dexter Kozen

Recap of Last Lecture

- Evaluation-based inference
  - Dynamic
  - Can deal with programs with unbounded loops

Likelihood Weighting

- A form of importance sampling where the proposal is the prior

$$\mathbb{E}_{q(X)}\left[\frac{p(X|Y)}{q(X)}r(X)\right] = \frac{1}{p(Y)}\mathbb{E}_{q(X)}\left[\frac{p(Y,X)}{q(X)}r(X)\right]$$
$$\simeq \frac{1}{p(Y)}\frac{1}{L}\sum_{l=1}^{L} W^l r(X^l),$$

$$W^l = \frac{p(Y,X^l)}{q(X^l)} = \frac{p(Y|X^l)p(X^l)}{p(X^l)} = p(Y|X^l)$$

If we use $p(X^l)$ as the proposal distribution

Y are observed/conditioned variables

Likelihood Weighting: Variants

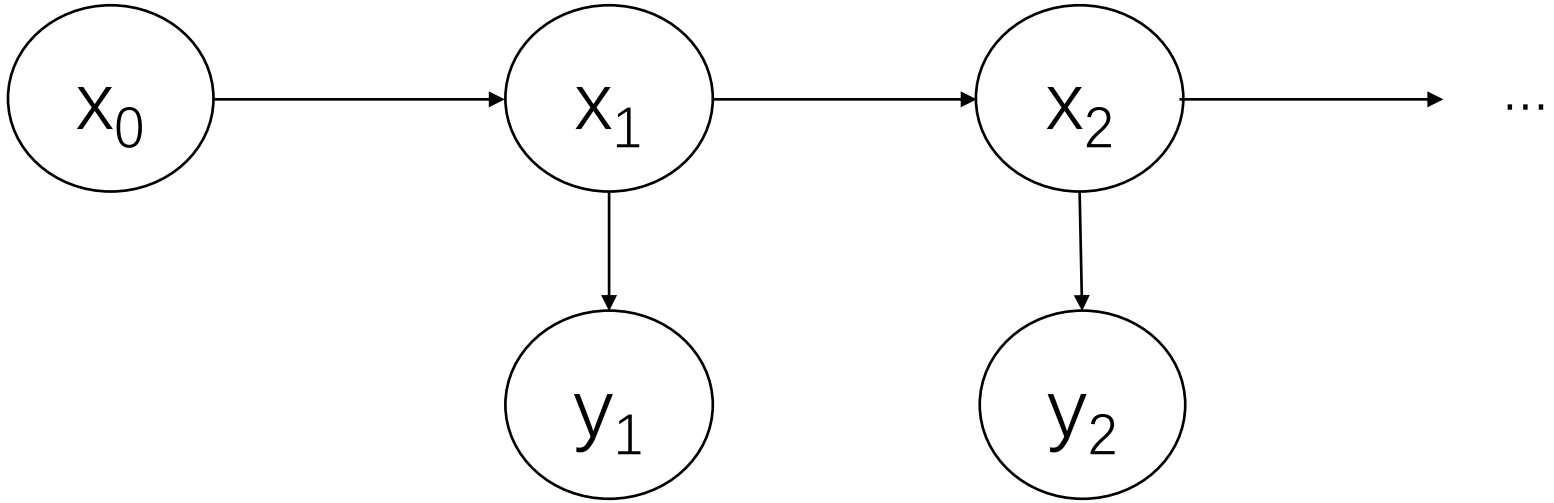
- Naïve Metropolis Hasting (draw random traces)

- Single-site proposal (try to only chance one variable at a time)

Sequential Monte Carlo

- In probabilistic programming, sample a high-dimensional distribution by sampling a sequence of lower dimensional distributions
- Also called particle filters
- Used in signal processing and probabilistic inference

SMC: Problem Statement



Given

$p(x_0)$ and
$p(x_t|x_{t-1})$ and
$p(y_t|x_t)$ and
Observations $y_{1:t}$

Estimate

$p(x_{0:t}|y_{1:t})$ or
$p(x_t|y_{1:t})$ or
$I(f_t) = E_{p(x_{0:t}|y_{1:t})}[f_t(x_{0:t})] = \int f_t(x_{0:t})p(x_{0:t}|y_{1:t})dx_{0:t}$

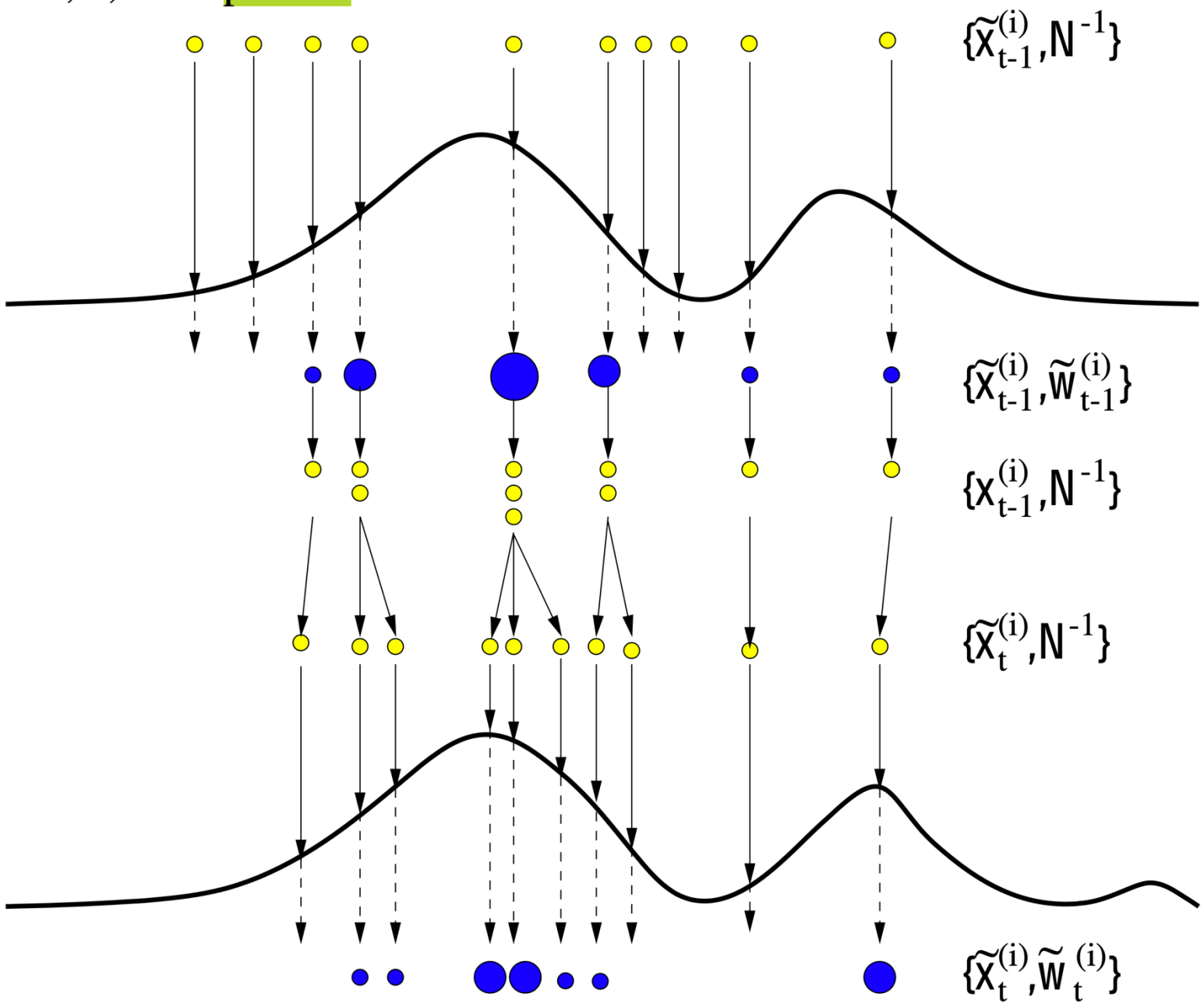
SMC: Main Ideas

- Sample on the Markov chain:

$$\pi\left(\mathbf{x}_{0:t}\middle|\,\mathbf{y}_{1:t}\right)=\pi\left(\mathbf{x}_{0:t-1}\middle|\,\mathbf{y}_{1:t-1}\right)\pi\left(\mathbf{x}_{t}\middle|\,\mathbf{x}_{0:t-1},\mathbf{y}_{1:t}\right).$$

- Reweight the samples using importance sampling

- Throw away the samples (particles) with low probabilities



From "An Introduction to Sequential Monte Carlo Methods" by Arnaud Doucet, Nando De Freitas, and Neil Gordon

SMC: Bootstrap Filter

Assume the proposal distribution is $p(x_{1:t})$

1. Initialization. T = 0
   - For i = 1,…,N, sample $x_0^{(i)} \sim p(x_0)$ and set $t = 1$
2. Importance sampling step.
   - For sample $\tilde{x}_t^{(i)} \sim p(x_t | \tilde{x}_{t-1}^{(i)})$ and set $(\tilde{x}_{0:t-1}^{(i)}, \tilde{x}_t^{(i)})$.
   - For i = 1,…,N, evaluate the importance weights.
   - Normalize the importance weights
3. Selection step
   - Resample with replacement N particles from the current particles according to importance weights
   - Set $t \rightarrow t + 1$

Question 1

- Consider the program x = unform(0, 1); y = gaussian(x, 1). Suppose the current trace is x = 0.5, y = 1. Now we want to change y, what is p(y) that we‘re sampling from?

- What if we want to change x?

Question 2

- Sequential Monte Carlo can be see as a variant of importance sampling. Is the statement right?

Question 3

- What would happen if we don't throw away particles in sequential Monte Carlo?

This Class

- Measure theory

- Semantics of probabilistic programming

This Class

- Measure theory

- Semantics of probabilistic programming

Motivations

- In order to reason about properties of a program, we need formal tools

- Example questions
  - Is the postcondition satisfied?
  - Does this program halt on all inputs?
  - Does it always halt in polynomial time?

Motivations

- In order to reason about properties of a program, we need formal tools

- Example questions
  - What is the probability that the postcondition is satisfied?
  - What is the probability that this program halts on all inputs?
  - What is the probability that it halts in polynomial time?

Motivations

- When designing a language, rigorous semantics is needed to guarantee its correctness

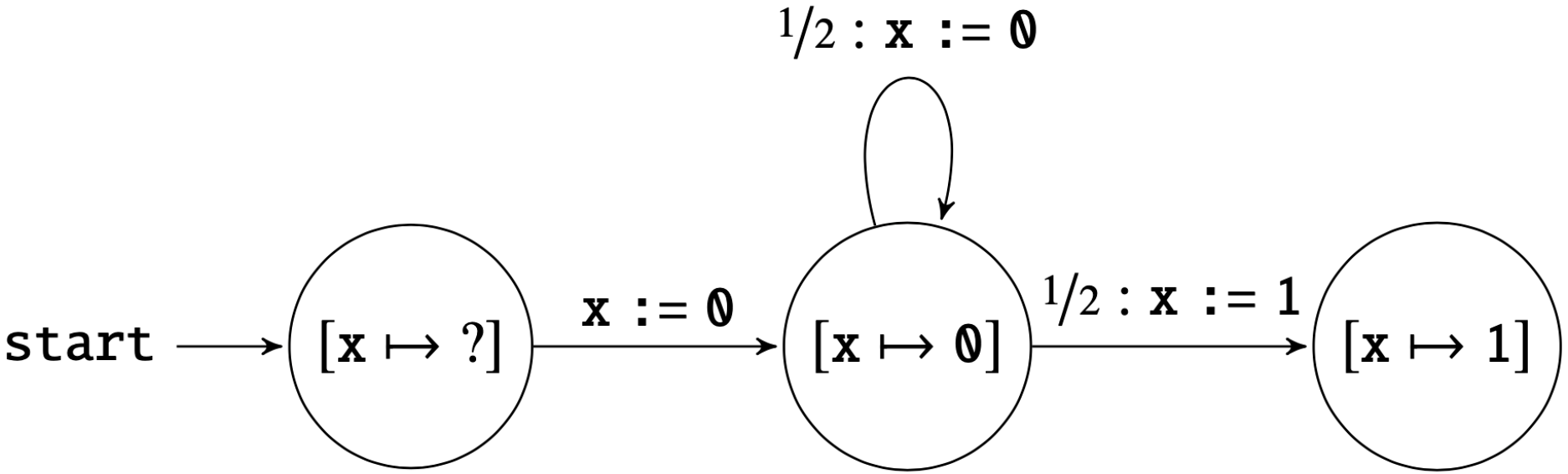
- Example that didn't have rigorous semantics: Javascript
  - https://javascriptwtf.com

Examples

We can decompose the semantics of a program into semantics of statements

```
x :=0
while x == 0 do
        x:=coin()
```

What is the probability that It runs through n iterations?
What is the expected number of iterations?
What is the probability that the program halts?



Examples

```
main{
        u:=0;
        v:=0;
        step(u,v);
        while u!=0 || v!=0 do
                step(u,v)
}

step(u,v){
        x:=coin();
        y:=coin();
        u:=u+(x-y);
        v:=v+(x+y-1)
}
```

What is the probability that the program halts?

The program is a two-dimensional random walk. According to probability theory, the probability that it returns to the origin is 1.

By relating to concepts in probabilities, we can simplify the reasoning

Examples

```
i:=0;
n:=0;
while i<1e9 do
        x:=rand();
        y:=rand();
        if (x*x+y*y) < 1 then n:=n+1;
        i:=i+1
i:=4*n/1e9;
```

What does this program compute?

How to reason about it?

Measure Theory
The mathematical foundation of probabilities and integration

Uniform(0,1) is called a *Lebesgue measure*

Measure Theory

- Measures: generalization of concepts like length, area, or volume

- We will talk about
  - What is a measurable space
  - Measures on measurable spaces
  - Rich structures of spaces of measures

Measure Example: Length

- What subsets of R can meaningfully be assigned a length?

- What properties should the length function l satisfy?

Measure Example: Length

$$\ell([a_1, b_1] \cup [a_2, b_2]) = \ell([a_1, b_1]) + \ell([a_2, b_2]) = (b_1 - a_1) + (b_2 - a_2). \qquad b_1 < a_2$$

$$\ell\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{i=1}^{n} \ell(A_i). \quad A_i \text{ and } A_j \text{ are disjoined . } l \text{ is called additive}$$

$$\ell\left(\bigcup_{i=0}^{\infty} A_i\right) = \sum_{i=0}^{\infty} \ell(A_i). \quad A_i \text{ and } A_j \text{ are disjoined . The set is countable.}$$
$$l \text{ is called countably additive or } \sigma - \text{additive}$$

$l(R) = \infty$, but we are only going to talk about finite measures

$$\ell(B \setminus A) = \ell(B) - \ell(A)$$ Domain should be closed under complementation

Measure Example: Length

- Can we extend the domain of length l to all subsets of R?

- No. Counterexample: Vitali sets
  - $V \subseteq [0,1]$, such that for each real number r, there exists exactly one number $v \in V$ such that $v - r$ is rational
  - Let $q_1, q_2, \ldots$ be the rational numbers in $[-1,1]$, construct sets $V_k = V + q_k$
  - $[0,1] \subseteq \bigcup_k V_k \subseteq [-1,2]$
  - $l(V_k) = l(V)$, and there infinitely many V_k

- l is called the *Lebesgue measure* on real numbers

Measurable Spaces and Measures

- $(\mathbf{S}, \mathbf{B})$ is a measurable space
  - $\mathbf{S}$ is a set
  - $\mathbf{B}$ is a σ-algebra on $\mathbf{S}$, which is a collection of subsets of $\mathbf{S}$
    - It contains $\emptyset$
    - Closed under complementation in $\mathbf{S}$
    - Closed under countable union
  - The elements of $\mathbf{B}$ are called measurable sets

- If $\mathbf{F}$ is a collection of subsets of $\mathbf{S}$, $\sigma(\boldsymbol{F})$ is the smallest σ-algebra containing $\mathbf{F}$, or $\sigma(\mathcal{F}) \triangleq \bigcap\{\mathcal{A} \mid \mathcal{F} \subseteq \mathcal{A} \text{ and } \mathcal{A} \text{ is a } \sigma\text{-algebra}\}$. We say (S, $\sigma(\boldsymbol{F})$) is generated by $\mathbf{F}$.

Measurable Functions

- $(\boldsymbol{S}, \boldsymbol{B_S})$ and $(\boldsymbol{T}, \boldsymbol{B_T})$ are measurable spaces. A function $f: \boldsymbol{S} \to \boldsymbol{T}$ is measurable if $f^{-1}(\boldsymbol{B}) = \{x \in S | f(x) \in B\}$ for every $\boldsymbol{B} \in \boldsymbol{B_T}$ is a measurable subset of S

Example: $$\chi_B(s) = \begin{cases} 1, & s \in B, \\ 0, & s \notin B. \end{cases}$$

Measures: Definitions

- A signed (finite) measure on $(\boldsymbol{S}, \boldsymbol{B})$ is a countably additive map $\mu: \boldsymbol{B} \rightarrow \boldsymbol{R}$ such that $\mu(\emptyset) = 0$
- Positive signed measure: $\mu(A) \geq 0$ for all $A \in \boldsymbol{B}$
- A positive measure is a probability measure if $\mu(S) = 1$
- …is a subprobability measure if $\mu(S) \leq 1$

Measures: Definitions

- If $\mu(B) = 0$, then B is a μ-nullset

- A property is said to hold μ-almost surely (everywhere) if the sets of points on which it does not hold is contained in nullset

- In probability theory, measures are sometimes called distributions

Measures: Discrete Measures

- For $s \in S$, the Dirac measure, or Dirac delta, or point mass on s:

$$\delta_s(B) = \begin{cases} 1, & s \in B, \\ 0, & s \notin B. \end{cases}$$

- A measure is discrete if it is a countable weighted sum of Dirac measures
  - If the weights add up to one, then it is a discrete probability measure

- Continuous measure: $\mu(\{s\}) = 0$ for all singleton sets {s} in $\boldsymbol{B}$ of $(\boldsymbol{S}, \boldsymbol{B})$

Measures: Pushforward Measure and Lebesgue Integration

- Given $f: (\boldsymbol{S}, \boldsymbol{B_S}) \to (\boldsymbol{T}, \boldsymbol{B_T})$ measurable, a measure μ on $\boldsymbol{B_S}$, the **pushfoward measure** $\mu(f^{-1}(B))$ on $\boldsymbol{B_T}$ is defined as

$$f_*(\mu)(B) = \mu(f^{-1}(B)), \quad B \in \mathcal{B}_T .$$

- **Lebesgue integration**: given $(\boldsymbol{S}, \boldsymbol{B})$, $\mu: \boldsymbol{B} \to \boldsymbol{R}$, $f: \boldsymbol{S} \to \boldsymbol{R}$, where $\mathrm{m} < f < M$

$$\int f \, d\mu = \lim_{n \to max} \sum_{i=0}^{n} f(s_i)\mu(B_i)$$

where $B_0, .., B_n$ is a measurable partition of $\boldsymbol{S}$, and the value of f does not vary more than $(M - m)/n$ in any B_i and $s_i \in B_i$

Measures: Absolute Continuity

- Given two measures μ and ν, we say μ is absolutely continuous with respect to ν for all measurable sets B iff $\nu(B) = 0 \Longrightarrow \mu(B) = 0$
  - $\mu \ll \nu$

Theorem 1.1 (Radon–Nikodym) *Let μ, ν be two finite measures on a measurable space $(S, \mathcal{B})$ and assume that μ is absolutely continuous with respect to ν. Then there exists a measurable function $f : S \to \mathbb{R}$ defined uniquely up to a μ-nullset such that*

$$\mu(B) = \int_B f \, d\nu.$$

The function f is called the Radon–Nikodym derivative *of μ with respect to ν.*

Measures: More on Radon-Nikodym

- Not related to semantics, but one pillar of the probability theory
- f is called the Radon-Nikodym derivative. One example is density function
- Extends probability masses and probability measures to measures over arbitrary set
- Example: μ: $gaussain, v$: $Lebesgue\ measure\ on\ R$

Products of Measurable Spaces

- Given $(\boldsymbol{S_1}, \boldsymbol{B_1})$ and $(\boldsymbol{S_2}, \boldsymbol{B_2})$, their product is $(\boldsymbol{S_1} \times \boldsymbol{S_2}, \boldsymbol{B_1} \otimes \boldsymbol{B_2})$ where
$$\boldsymbol{B_1} \otimes \boldsymbol{B_2} = \sigma(\{B_1 \times B_2 \mid B_1 \in \boldsymbol{B_1}, B_2 \in \boldsymbol{B_2}\})$$

- A measure on $(\boldsymbol{S_1} \times \boldsymbol{S_2}, \boldsymbol{B_1} \otimes \boldsymbol{B_2})$ is sometimes called a joint distribution

$$(\mu_1 \otimes \mu_2)(B_1 \times B_2) \triangleq \mu_1(B_1)\mu_2(B_2).$$

- A special case

μ_1 and μ_2 are independent

Markov Kernels

- Given $(\boldsymbol{S}, \boldsymbol{B_S})$ and $(\boldsymbol{T}, \boldsymbol{B_T})$, $P: \boldsymbol{S} \times \boldsymbol{B_T} \to \boldsymbol{R}$ is called a Markov kernel if
  - For fixed $A \in \boldsymbol{B_T}$, the map $\lambda s. P(s, A) \to \boldsymbol{R}$ is a measurable function on $(\boldsymbol{S}, \boldsymbol{B_S})$
  - For fixed $s \in \boldsymbol{S}$, the map $\lambda A. P(s, A) \to \boldsymbol{R}$ is a probability measure on $(\boldsymbol{T}, \boldsymbol{B_T})$

- Composition of two Markov kernels
  - Given $P: S \to T$, $Q: T \to U$ $(P \,;\, Q)(s, A) = \int_{t \in T} P(s, dt) \cdot Q(t, A).$

- Given μ on $\boldsymbol{B_S}$, its push forward under the Markov Kernel P is

$$P_*(\mu)(B) = \int_{s \in S} P(s, B)\ \mu(ds).$$

More on Markov Kernels

- $(\boldsymbol{S}, \boldsymbol{B_S})$: x = … (x>0)
- $(\boldsymbol{T}, \boldsymbol{B_T})$: y = uniform(0,x)
- Markov kernel $P(x, \bigcup_{i=1}^{i=M}[a_i, b_i]) = \sum_{i=1}^{i=M} length([a_i, b_i] \cap [0, x])/x$

More on Markov Kernels

- $(\boldsymbol{S}, \boldsymbol{B_S})$: x = … (x>0)
- $(\boldsymbol{T}, \boldsymbol{B_T})$: y = uniform(0,x)
- $(\boldsymbol{T}, \boldsymbol{B_T})$: z = uniform(0,y)
- Composition: $(P;Q)(x,[0,z]) = \int_{y\in[0,\infty]} P(x,dy) * Q(y,[0,z])$

z < x

$$= \int_{y\in[0,x]} \frac{dy}{x} * \frac{length([0,z]\cap[0,y])}{y}$$

$$= \int_{y\in[0,z]} \frac{dy}{x} * \frac{y}{y} + \int_{y\in[z,x]} \frac{dy}{x} * \frac{z}{y} = \frac{z}{x} + \frac{z}{x}(lnx - lnz)$$

More on Markov Kernels

- $(\boldsymbol{S}, \boldsymbol{B_S})$: x = uniform(0.1, 1.1) $\mu([a,b]) = \text{length}([\text{a}, \text{b}] \cap [0.1,1.1])$
- $(\boldsymbol{T}, \boldsymbol{B_T})$: y = uniform(0,x)
- Markov kernel $P(x, \bigcup_{i=1}^{i=M}[a_i, b_i]) = \sum_{i=1}^{i=M} length([a_i, b_i] \cap [0, x])/x$
- μ's pushforward under P is

$$P_*(\mu)(B_T) = \int_{x \in [0.1,1.1]} B_T \cap [0, x] * \mu(dx)$$

More on Markov Kernels

- We can use Markov kernels to define the meanings of statements

- A term can be seen as a Markov kernel that links the input variables (can be a distribution) with the output distribution

Spaces of Measures

- We now talk about the structures of the spaces of measures
  - This will allow us to talk about general properties of measures

- $\boldsymbol{M(S,B)}$ or $\boldsymbol{MS}$ is the set of all finite, signed measures on a measurable set $\boldsymbol{(S,B)}$

Vector Space Structure

- $\boldsymbol{MS}$ is always a real vector space

$$(\mu + \nu)(B) \triangleq \mu(B) + \nu(B) \qquad (a\mu)(B) \triangleq a\mu(B)$$

Normed Space Structure

- Every measure has a norm

$$\|\mu\| \triangleq \sup\left\{\sum_{i=1}^{n}|\mu(B_i)| : \{B_1,\ldots,B_n\} \text{ is a finite measurable partition of } S\right\}.$$

- For positive measures, $\big|\|\mu\|\big| = \mu(S)$

- A complete normed vector space is a Banach space

Order Structure

- Measures have a natural pointwise order: $\mu \leq \nu \; if \; \mu(B) \leq \nu(B), \forall B$
- Are two distinct probability measures comparable?
- The partial order is compatible with the vector space structure:
  - if $\mu \leq \nu$, then $\mu + \rho \leq \nu + \rho$; and
  - if $0 \leq a \in \mathbb{R}$ and $\mu \leq \nu$, then $a\mu \leq a\nu$.
- Additions and multiplications by a positive scalar are monotone

Order Structure

- The partial order defines a lattice

$$(\mu \vee \nu)(B) \triangleq \sup\{\mu(A \cap B) + \nu(A^c \cap B) \mid A \in \mathcal{B}\}$$

$$(\mu \wedge \nu)(B) \triangleq \inf\{\mu(A \cap B) + \nu(A^c \cap B) \mid A \in \mathcal{B}\}.$$

- Why do we care? It will be used to deal with loops

- $\mu^+ = \mu \vee 0, \quad \mu^{-1} = -\mu \vee 0, \mu = \mu^+ - \mu^-$, modulus $|\mu| = \mu^+ + \mu^-$

- The order is compatible with the norm: $|\mu| \leq |\nu| \Longrightarrow ||\mu|| \leq ||\nu||$

Order Structure

- $\boldsymbol{MS}$ is a Banach lattice:
  - A Banach space with a lattice structure that is compatible with both the linear and normed structures

- A Banach lattice is σ-order-complete if every countable order-bounded set of measures in $\boldsymbol{MS}$ has a supremum in $\mathbf{MS}$

- Every measure space is σ-order-complete

Order Structure

- For any measure set **MS,** every countable order-bounded set of measures in $\boldsymbol{MS}$ has a supremum in **MS**

- This will help us deal with loops
  - Every iteration can be seeing as joining measures
  - The measures are bounded
  - They will converge to the supremum

Operators

- Since spaces of measures are vector spaces, we can do linear algebra
- Linear operator $T: T(x) + T(y) = T(x + y), T(ax) = aT(x)$
- We are mostly interested in operators that send probability measures to subprobability measures (conditional probabilities)

Summary

- Measure theory is the theory about measures (generalization of length, area, volume…)
  - Foundation of probabilities and integration

- Measurable space
- Measures: distribution, state of a program
- Markov kernels: allows us to model statements
- The space of measures on a given measure space is a σ-order-complete Banach lattice

Outline

- Measure theory

- Semantics of probabilistic programming

A Simple Imperative Language

- Highly simplified version

- Enough to explain the core concepts

Syntax

- Deterministic terms (expressions)
- Terms (Deterministic + Probabilistic)
- Tests (expression that evaluate to Booleans)
- Programs

Syntax – Deterministic Terms

(i) Deterministic terms:

$$
\begin{aligned}
d &::= a && a \in \mathbb{R}, \text{ constants} \\
&\mid x && x \in \text{Var}, \text{ a countable set of variables} \\
&\mid d \ \text{op} \ d && \text{op} \in \{+, -, *, \div\}
\end{aligned}
$$

Syntax - Terms

(ii) Terms:

$$
\begin{array}{lll}
t ::= d & & d \text{ a deterministic term} \\
\quad \mid \texttt{coin()} \mid \texttt{rand()} & & \text{sample in } \{0, 1\} \text{ and } [0, 1], \text{ respectively} \\
\quad \mid t \ \text{op} \ t & & \text{op} \in \{+, -, *, \div\}
\end{array}
$$

Syntax - Tests

(iii) Tests:

$$
\begin{array}{lll}
b ::= \texttt{true} \mid \texttt{false} & & \\
\quad \mid d == d \mid d < d \mid d > d & & \text{comparison of deterministic terms} \\
\quad \mid b \,\&\&\, b \mid b \mid\mid b \mid \texttt{!}\, b & & \text{Boolean combinations of tests}
\end{array}
$$

Syntax - Program

(iv) Programs:

e ::= `skip`	
| x `:=` t	assignment
| e `;` e	sequential composition
| `if` b `then` e `else` e	conditional
| `while` b `do` e	`while` loop

Syntax - Example Program

```
if coin() == 1 then
        x := rand() * 5
else
        x := 6
if x > 4.5 then
        y := coin() + 2
else
        y := 100
```

Operational Semantics

- Model the step-by-step executions of a program on a machine

- Tracks the memory-state
  - Values assigned to each variable
  - Values assigned to each random generator
  - A stack of instructions

Denotational Semantics

- Model all possible executions together
- States: probability distribution over memory states
- $\frac{1}{2}s[x \mapsto 0] + \frac{1}{2}s[x \mapsto 1]$

Denotational Semantics: Introduction

- State s can be identified with the Dirac measure σ_s, then the semantics of x:=coin() can be viewed as $\sigma_s \to \frac{1}{2} s[x \mapsto 0] + \frac{1}{2} s[x \mapsto 1]$

- In general, a program is interpreted as an operator mapping probability distributions to (sub)probability distributions

Denotational Semantics: Definition

- Assume there are n real variables, then a state is a distribution on R^n

- A program e: $MR^n \rightarrow MR^n$
  - An operator called a state transformer

Denotational Semantics: Terms

$$[\![r]\!](a_1,\ldots,a_n) = \delta_r,\ r \in \mathbb{R}$$

$$[\![x_i]\!](a_1,\ldots,a_n) = \delta_{a_i}$$

$$[\![\texttt{coin()}]\!](a_1,\ldots,a_n) = \frac{1}{2}\delta_0 + \frac{1}{2}\delta_1$$

$$[\![\texttt{rand()}]\!](a_1,\ldots,a_n) = \lambda$$

$$[\![t_1 \text{ op } t_2]\!](a_1,\ldots,a_n) = \text{op}_*([\![t_1]\!](a_1,\ldots,a_n) \otimes [\![t_2]\!](a_1,\ldots,a_n))$$

The denotational semantics of any term is a Markov kernel $\boldsymbol{R^n \to MR}$

$op \in \{+,-,\times,\div\}, \lambda$ *is the Lebesgue measure on* $[0,1]$, op_* *denotes push forward*

$$\text{op}_*([\![t_1]\!](a_1,\ldots,a_n) \otimes [\![t_2]\!](a_1,\ldots,a_n))(B)$$

$$= [\![t_1]\!](a_1,\ldots,a_n) \otimes [\![t_2]\!](a_1,\ldots,a_n)\left\{(u,v) \in \mathbb{R}^2 \mid u \text{ op } v \in B\right\}.$$

Denotational Semantics: Tests

$$[\![t_1 == t_2]\!] = \{(a_1, \ldots, a_n) \in \mathbb{R}^n \mid [\![t_1]\!](a_1, \ldots, a_n) = [\![t_2]\!](a_1, \ldots, a_n)\}$$

$$[\![t_1 < t_2]\!] = \{(a_1, \ldots, a_n) \in \mathbb{R}^n \mid [\![t_1]\!](a_1, \ldots, a_n) < [\![t_2]\!](a_1, \ldots, a_n)\}$$

$$[\![t_1 > t_2]\!] = \{(a_1, \ldots, a_n) \in \mathbb{R}^n \mid [\![t_1]\!](a_1, \ldots, a_n) > [\![t_2]\!](a_1, \ldots, a_n)\}$$

$$[\![b_1 \;\&\&\; b_2]\!] = [\![b_1]\!] \cap [\![b_2]\!]$$

$$[\![b_1 \;||\; b_2]\!] = [\![b_1]\!] \cup [\![b_2]\!]$$

$$[\![!b]\!] = [\![b]\!]^c.$$

The denotational semantics of any test is a measurable subset of R^n

Denotational Semantics: Tests

- How are semantics of tests used?

Each measurable set $B \subseteq R^n$ defines a linear operator:

$$T_B(\mu)(C) = \mu(B \cap C).$$

- Tests usually send states to sub-probability distributions

Denotational Semantics: Program

Programs are interpreted as operators $\mathcal{M}\mathbb{R}^n \to \mathcal{M}\mathbb{R}^n$

(i) $[\![\texttt{skip}]\!] = \mathrm{Id}_{\mathcal{M}\mathbb{R}^n} : \mathcal{M}\mathbb{R}^n \to \mathcal{M}\mathbb{R}^n$, the identity operator $\mu \mapsto \mu$.

(ii) Assignments:

Given such a term t and an index $1 \leq i \leq n$, we define the Markov kernel $F_t^i : \mathbb{R}^n \to \mathcal{M}\mathbb{R}^n$ as

$$F_t^i(x_1, \ldots, x_n) = \delta_{x_1} \otimes \ldots \otimes \delta_{i-1} \otimes [\![t]\!](x_1, \ldots, x_n) \otimes \delta_{x_{i+1}} \otimes \ldots \otimes \delta_{x_n}$$

Using this definition, we define $[\![x_i := t]\!]$ as the operator

$$[\![x_i := t]\!](\mu) = (F_t^i)_*(\mu)$$ (F_t^i) is the push forward of the Markov kernel

Denotational Semantics: More on Assignments

$$
\begin{aligned}
&[\![x_i := r]\!](\mu)(B_1 \times \cdots \times B_n) \\
&\quad = \int_{\mathbb{R}^n} \delta_{x_1} \otimes \cdots \otimes \delta_{x_{i-1}} \otimes \delta_r \otimes \delta_{x_{i+1}} \otimes \cdots \otimes \delta_{x_n}(B_1 \otimes \cdots \otimes B_n)\, d\mu \\
&\quad = \int_{\mathbb{R}^n} \delta_{x_1}(B_1) \cdots \delta_{x_{i-1}}(B_{i-1}) \delta_r(B_i) \delta_{x_{i+1}}(B_{i+1}) \cdots \delta_{x_n}(B_n)\, d\mu \\
&\quad = \mu(B_1 \times \cdots \times B_{i-1} \times \mathbb{R} \times B_{i+1} \times \cdots \times B_n) \delta_r(B_i).
\end{aligned}
$$

$$
\begin{aligned}
&[\![x_i := \texttt{coin()}]\!](\mu)(B_1 \times \cdots \times B_n) \\
&\quad = \mu(B_1 \times \cdots \times B_{i-1} \times \mathbb{R} \times B_{i+1} \times \cdots \times B_n) \left(\frac{1}{2}\delta_0 + \frac{1}{2}\delta_1\right)(B_i).
\end{aligned}
$$

Denotational Semantics: Program

(iii) $[\![e_1 \texttt{ ; } e_2]\!] = [\![e_2]\!] \circ [\![e_1]\!]$, the usual composition of operators.

(iv) $\texttt{if } b \texttt{ then } e_1 \texttt{ else } e_2$ is the operator defined by

$$[\![\texttt{if } b \texttt{ then } e_1 \texttt{ else } e_2]\!] = [\![e_1]\!] \circ T_{[\![b]\!]} + [\![e_2]\!] \circ T_{[\![b]\!]^c}$$

Denotational Semantics: Program

(v) To define the semantics of `while` loops, we use the equivalence of the following two programs,

$$\texttt{while}\ b\ \texttt{do}\ e \qquad\qquad \texttt{if}\ b\ \texttt{then}\ (e\ ;\ \texttt{while}\ b\ \texttt{do}\ e)\ \texttt{else skip},$$

$$[\![\texttt{while}\ b\ \texttt{do}\ e]\!] = [\![\texttt{while}\ b\ \texttt{do}\ e]\!] \circ [\![e]\!] \circ T_{[\![b]\!]} + T_{[\![b]\!]^c}$$

Which is the least fixed point of $\tau(S) = S \circ [\![e]\!] \circ T_{[\![b]\!]} + T_{[\![b]\!]^c}$

$$[\![\texttt{while}\ b\ \texttt{do}\ e]\!] = \bigvee_{n\geq 0} \tau^n(0) = \sum_{k=0}^{\infty} T_{[\![b]\!]^c} \circ \left([\![e]\!] \circ T_{[\![b]\!]}\right)^k$$

Constructable because norm-bounded directed sets have suprema and that τ preserves such suprema

Denotational Semantics: Program

Theorem 1.5 *The denotational semantics of any program given by the syntax of Section 1.3.1 is a positive operator of norm at most one.*

Denotational Semantics: Examples

$$[\![\texttt{x := 0 ; while x == 0 do x := coin()}]\!] : \mathcal{M}\mathbb{R} \to \mathcal{M}\mathbb{R}$$

$$[\![\texttt{x := 0}]\!] : \mathcal{M}\mathbb{R} \to \mathcal{M}\mathbb{R} \qquad \mu \mapsto \mu(\mathbb{R})\delta_0,$$

$$[\![\texttt{x := coin()}]\!] : \mathcal{M}\mathbb{R} \to \mathcal{M}\mathbb{R} \qquad \mu \mapsto \mu(\mathbb{R})(\tfrac{1}{2}\delta_0 + \tfrac{1}{2}\delta_1)$$

What about the while loop?

Denotational Semantics: Examples

$$[\![\texttt{x := 0 ; while x == 0 do x := coin()}]\!] : \mathcal{M}\mathbb{R} \to \mathcal{M}\mathbb{R}$$

$$\tau^0(0)(\mu) = 0$$

$$\tau^1(0)(\mu) = T_{\{0\}^c}(\mu) = \mu(- \cap \{0\}^c)$$

$$\tau^2(0)(\mu) = \left(T_{\{0\}^c} + T_{\{0\}^c} \circ [\![\texttt{x := coin()}]\!] \circ T_{\{0\}}\right)(\mu)$$
$$= \mu(- \cap \{0\}^c) + \frac{\mu(\{0\})}{2}\delta_1.$$

$$\tau^k(0)(\mu) = \mu(- \cap \{0\}^c) + \sum_{i=1}^{k-1} \frac{\mu(0)}{2^i}\delta_1$$
$$= \mu(- \cap \{0\}^c) + (1 - 2^{-(k-1)})\mu(\{0\})\delta_1,$$

It's limit is $\mu(- \cap \{0\}^c) + \mu(\{0\})\delta_1$

$$[\![\texttt{x := 0 ; while x == 0 do x := coin()}]\!] : \mathcal{M}\mathbb{R} \to \mathcal{M}\mathbb{R} \qquad \mu \mapsto \mu(\mathbb{R})\delta_1.$$

More Examples

- Refer to the supplementary reading
- https://www.cambridge.org/core/services/aop-cambridge-core/content/view/A7964205E44B5234A78C661192E294E1/9781108488518c1_1-42.pdf/semantics_of_probabilistic_programming_a_gentle_introduction.pdf

Left Issues: Initial State in Denotational

- We can assume that each variable is assigned to a default value

- A Dirac measure

Left Issues: Condition

- Operational semantics: model the traces that fails to satisfies the condition as erroneous traces

- Denotational semantics: similar to tests, but need to renormalizes the measures

Left Issues: Non-Terminating Executions

$$\texttt{while true do } e.$$

- Operational semantics: infinite trace

- Denotational:
  - Can declare infinite traces are invalid
  - Can still compute some least fixed point (more involved)